

AI-Wrevolusi

Iteration 2 Risk Analysis Report

FIT5120 Industry Experience | 17 September 2026

Overview

Iteration 2 gives AI-Wrevolusi more ways to support users with work tasks and learning plans. People can save account workspaces, track learning and return to stored AI exchanges. These improvements also create more opportunities for personal information to be exposed, changed or kept longer than expected.

Workspace saves check ownership and revision numbers. Several AI requests have input limits and structured responses. However, the review also found inconsistent session controls, unrestricted login attempts in the reviewed application code, external AI fallback behaviour and incomplete evidence for recovery. These are the main reasons several current risks remain High.

This report explains possible harm and proposed action. The scenarios do not mean that an attack or breach has happened.

Scope and evidence

The assessment compares the iteration-1-release snapshot at 2499736 with the current checkout at 57d0d78 on kuan/design-prototype. The snapshots sit on different lines of Git history, so the comparison does not represent one straight release sequence. [C1]

Evidence comes from current source, selected local tests and isolated checks. Hosted settings, real provider behaviour and live database permissions were not verified. Suggested owners are roles for the team to assign. No person has accepted responsibility or risk through this report.

Why Iteration 2 changes the risk picture

Change	Added responsibility
Account workspace synchronization	Prevent account mixing, conflicting saves and unwanted local copies.
Learning progress and daily briefs	Protect a growing record of personal activity.
External AI matching and assistance	Control where task text goes and explain the limits of generated advice.
Saved Task Assist exchanges	Define retention and deletion for questions, notes and answers.

Password hashing, cookie sessions and the public occupation-write route already existed in Iteration 1. Their remaining weaknesses are carried forward. [C1, C4, C14]

1. Ethical, privacy and legal considerations

Advice can influence real decisions

An unclear exposure score could make someone believe their job will disappear. A confident skill suggestion could lead them towards an unsuitable course. The system should explain that these results support exploration and do not predict an individual's employment outcome.

A valid response format does not make the advice correct. The current gateway checks response structure and candidate IDs, but some candidates come from the caller. A confirmed skill-finalizer gap also keeps the original suggestions when every evidence phrase fails its text check. This weakens the explanation shown to the user. The proposed response is to reject unsupported evidence, explain uncertainty and preserve the user's ability to review suggestions. [C7, C15]

Work descriptions can identify people

A username-only account reduces the need for a real email address. It does not make the whole workspace anonymous. Task descriptions, workplace routines and free-text notes may identify an employer, customer or colleague. The team should encourage general descriptions and avoid asking for names, internal documents or customer information that the feature does not need.

Task Assist stores the first question and reply automatically. The response's confirmation flag concerns review of the guidance. It is not evidence of permission before storage. Clear wording should explain both external processing and saved history before the user submits a question. [C3, C8]

Legal position needs confirmation

Malaysia's data protection authority publishes the Personal Data Protection Act 2010 portal and guidance on cross-border transfers. These are relevant starting points for reviewing a Malaysian-facing service. [Act 709 portal](#) and [cross-border transfer guidance](#).

The operator, applicable scope and provider jurisdictions have not been confirmed. This report therefore does not claim legal compliance or decide which duties apply. Before wider use, the proposed product and deployment owners should document who operates the service, where information is processed, provider terms, retention and the process for handling user requests.

2. How the risks are rated

Scores help choose work. They are judgement-based, not measured probabilities, and consider the evidence in this checkout.

Inherent risk means the exposure before relevant controls. **Current remaining risk** means the exposure after considering existing controls and known gaps. Only current remaining scores are given here. There is no reliable measured baseline for assigning earlier scores or claiming a numerical reduction since Iteration 1.

Score	Likelihood in the present scope	Impact if the event occurs
1	Rare and requires unusual conditions	Small inconvenience with no meaningful personal-data effect
2	Unlikely, with useful barriers present	Limited disruption or a readily corrected record
3	Possible through a realistic failure or misuse	Personal work exposed, altered or temporarily unavailable
4	Likely under repeated requests or common misuse	Account access, significant disclosure or extended disruption
5	Very likely without further action	Widespread compromise or severe service damage

Score = likelihood × impact. The bands are Low at 1–4, Medium at 5–9, High at 10–16 and Critical at 17–25. High scores do not prove an attack. Hosted controls remain unverified.

HIGH | 6 risks

R1 Configuration		10
R2 Sessions		12
R3 Password attempts		12
R5 External AI disclosure		12
R9 Excess requests		12
R11 Detection and recovery		12

MEDIUM | 6 risks

R4 Browser copies		9
R6 Wrong-account access		8
R7 Browser requests		9
R8 Misleading results		9
R10 Retention and sharing		9
R12 Occupation writes		9

Current remaining score = likelihood × impact. These are review judgements, not measured probabilities.

Six risks currently rate High. Five score 12 and receive detailed treatment in Section 4. R1 scores 10, but its potential impact makes configuration verification a separate release condition. Medium risks still require action, especially where shared computers or personal work text are involved.

Password hashing helps after a database leak. Login limits address repeated online attempts. Each control must fit the scenario.

3. Current risk register

All scores below are current remaining scores. Owner roles are proposed. No risk acceptance is recorded.

3.1 Identity, privacy and ownership

ID and scenario	Current control and remaining concern	L × I	Proposed treatment and owner
R1. Unsafe production configuration. A deployment uses development signing or cookie settings.	Settings are configurable, but a known development signing default has no production rejection guard. Hosted values were not checked. [C10]	$2 \times 5 = 10$ High	Reject unsafe production settings and verify hosted transport. Deployment owner.
R2. Session remains usable unexpectedly. A copied token survives logout or an inactive account retains access.	Cookies and refresh revocation exist. Older endpoints also return tokens in JSON, access checks omit is_active, and refresh consumption is not atomic. [C4]	$3 \times 4 = 12$ High	Align session responses, revocation and account-state checks. Backend owner.
R3. Repeated password attempts succeed. An attacker tries reused passwords or targets weak accounts.	Hashing is present. No application login throttle was found. Legacy registration schemas permit an empty password while username registration requires eight characters. [C3, C4]	$3 \times 4 = 12$ High	Apply one credential policy and shared login limits. Backend and deployment owners.
R4. Personal work remains on a shared device. A failed save blocks normal logout or a cache remains accessible.	Account-specific caches and save checks reduce mixing. They do not make browser storage private from the device user or same-origin script. [C3, C5]	$3 \times 3 = 9$ Medium	Allow local exit after a failed save and explain unsaved-work choices. Frontend owner.
R5. Workplace text reaches an unexpected AI recipient. A fallback receives a task after the main provider fails.	Payloads are structured, but external fallback is enabled by default and can work without the primary key. [C7, C8]	$3 \times 4 = 12$ High	Approve destinations, minimize fields and disclose processing. Product and backend owners.
R6. Wrong-account access or updates occur. A delayed or manipulated request uses another user's records.	Ownership, workspace revisions and session guards are present. Failing Task Assist contract tests and the absence of live database checks leave some behaviour unverified. [C3, C8, C9]	$2 \times 4 = 8$ Medium	Repair tests and exercise two-account isolation with PostgreSQL. Backend and test owners.

R6 is lower because direct ownership barriers are visible. Negative tests remain necessary to catch regressions.

3.2 Browser, reliability and information integrity

ID and scenario	Current control and remaining concern	L × I	Proposed treatment and owner
R7. Browser content or forged requests misuse a session. A hostile input or origin triggers an unwanted action.	Text rendering, an origin allow-list and SameSite cookies help. A dedicated CSRF check was not found in the reviewed backend. [C2, C4]	$3 \times 3 = 9$ Medium	Verify rendering, state-changing request checks and browser headers. Frontend and backend owners.
R8. Misleading AI or reference results influence a user. Unsupported evidence appears to justify a career suggestion.	Structured responses, candidate filtering and user review help. The skill finalizer can retain evidence that failed verification. [C7, C15]	$3 \times 3 = 9$ Medium	Reject unsupported evidence and test uncertain or misleading cases. AI and product owners.
R9. Excess requests exhaust capacity. Repeated public requests consume provider quota, memory or processing time.	A provider counter limits calls within one process. It is not a shared incoming-request quota, and separate skill routes bypass it. [C7]	$4 \times 3 = 12$ High	Add shared request limits, input bounds and cache eviction. Backend and deployment owners.
R10. Records or copies outlive their purpose. Old notes remain stored or a downloaded backup exposes personal plans.	Ownership protects server access, but a complete retention and deletion workflow was not verified. Downloads leave account protection. [C5, C8, C12, C13]	$3 \times 3 = 9$ Medium	Define retention, deletion and clear export previews. Product, backend and frontend owners.
R11. A failure is detected late or recovery loses work. The team cannot establish the impact or restore a useful state.	Error handling exists, but a complete security event trail and tested recovery procedure were not established by this review. [C12]	$3 \times 4 = 12$ High	Add purposeful events and rehearse isolated recovery. Deployment and backend owners.
R12. An unauthenticated caller adds occupation records. Repeated writes pollute a public legacy dataset.	POST /api/v1/occupations has no authentication dependency. It writes the legacy occupations table, not the official ref_occupations table. Hosted restrictions were not tested. [C14]	$3 \times 3 = 9$ Medium	Restrict creation to an authorized workflow or remove the public write path. Backend owner.

R8 includes harm from unreliable guidance. R12 concerns access control. The review does not show official reference-table modification or actual abuse.

4. Five highest-scoring priorities

4.1 R2: make session behaviour consistent

A user expects logout to end their access. The current operation clears cookies and revokes the presented refresh record, but access-token validation does not consult that record. A previously copied access token can remain usable until expiry. Legacy token responses and the missing inactive-user check add related paths for unexpected access. [C4]

Likelihood is Possible because these paths are reachable, although a token may need to be exposed first. Impact is High because sessions grant access to personal work.

The proposed backend owner should define session-ending behaviour, remove unnecessary token bodies from browser endpoints and enforce account status consistently. Refresh consumption should use a transaction-safe operation.

Acceptance evidence: browser session responses contain no tokens, inactive accounts are rejected, logout meets the chosen policy, and two simultaneous refreshes cannot both consume the same record in disposable PostgreSQL. **Early warning:** repeated refresh reuse, token use after recorded logout or unusual authentication failures. **Contingency:** restrict affected sessions, rotate exposed credentials where necessary and require fresh sign-in after the cause is fixed.

4.2 R3: protect login and registration together

An attacker does not need to break password hashing to try passwords taken from another service. The reviewed application has no login throttle, and the older registration schemas accept an empty password. The stricter username screen therefore does not establish one consistent account policy. [C3, C4]

The score is 12 because repeated attempts are realistic and account access has a high impact.

The proposed backend and deployment owners should apply the same password rules to every supported registration path and introduce shared limits across workers. Limits should slow abuse without allowing one attacker to lock another person out indefinitely.

Acceptance evidence: empty or short passwords are rejected on all supported paths, repeated attempts receive controlled responses and the limit remains effective across two workers. **Early warning:** bursts of failed logins and many accounts targeted from related requests. **Contingency:** apply temporary tighter limits and investigate affected accounts using event metadata, without logging attempted passwords.

4.3 R5: make external AI processing explicit

A missing primary API key does not guarantee local processing. The gateway can instead create a keyless fallback provider. After a primary failure, the same information can also be tried through the fallback. Task Assist sends task text, notes and the question, so an unexpected destination can matter even without a real name field. [C7, C8]

Likelihood is Possible because fallback is part of normal failure handling. Impact is High because users may type confidential workplace details. Code inspection cannot tell us the provider's actual retention practices or where its processing occurs.

The proposed product and backend owners should approve a short destination list and explain what information leaves the application before submission. Unapproved fallback routes should be disabled. Payloads should include only fields needed for the feature. A local option must be tested by checking that it makes no outbound request.

Acceptance evidence: dummy-data request capture shows the approved recipients and fields, failure follows the approved fallback policy and the user sees understandable disclosure. **Early warning:** calls to an unexpected host, new payload fields or increased fallback use. **Contingency:** disable the provider and its fallback, preserve minimal investigation records and show a clear local or unavailable result.

4.4 R9: limit requests at the application boundary

Several matching routes are public. Existing provider counters work within one provider object in one process. They do not identify a user, remain shared across workers or cover all skill-direction calls. A higher-level cache also has no eviction limit. These gaps make repeated requests a practical availability concern. [C7]

Likelihood is rated 4 because generating requests is easy and does not always require an account. Impact is 3 because the expected harm includes delayed guidance, exhausted quotas and service disruption. The review did not measure a production outage or financial loss.

The proposed backend and deployment owners should use shared incoming limits, bounded request sizes, bounded caches and suitable timeouts. Public exploration can remain available with proportionate limits.

Acceptance evidence: excess requests stop before provider use, limits work across two workers and sustained unique inputs do not grow the cache without bound. **Early warning:** rising response time, queue length, memory use and provider calls. **Contingency:** temporarily disable expensive generation while preserving saved work and a clear unavailable message.

4.5 R11: make detection and recovery usable

Even good prevention cannot guarantee that every dependency or request succeeds. If records disappear or an account is misused, the team needs enough information to understand what happened. It also needs a tested way to restore service. A schema for review events does not prove that important runtime actions create a complete audit history. [C12]

Likelihood is Possible because failures can arise from ordinary software changes and service outages. Impact is High because weak investigation and recovery can extend disruption or leave users unsure whether their work is intact.

The proposed deployment and backend owners should record purposeful events such as failed authentication, denied ownership checks and failed saves. Events should use request identifiers and safe account identifiers, without passwords, tokens or full task text. Recovery should be rehearsed with fictional records in a separate environment.

Acceptance evidence: a controlled event is visible to the responsible role and a restored database preserves tested relationships and ownership. Record what recent changes are missing. **Early warning:** failed saves, unexplained record differences and monitoring silence during a known failure. **Contingency:** restrict conflicting writes, preserve unsynced work and restore only after checking the recovered state.

5. What the tests establish

The selected frontend checks passed **33 tests**. They cover storage, ownership boundaries, account switching and related planning or Task Assist behaviour. They include helper execution and source-contract checks, rather than a complete browser security exercise. [T1]

The selected backend run produced **104 passes and 9 failures**. Some failures reflect changed skill-output expectations. Task Assist fixtures also use assumptions from an older UUID contract while the current API uses string task keys. These results leave verification gaps. They do not demonstrate nine live security breaches. [T2]

Six additional isolated observations confirmed both legacy token fields, HttpOnly cookies, acceptance of an inactive mocked user, empty-password acceptance in both legacy schemas and retained nonmatching skill evidence. No real account, database or provider was used. [T3]

The proposed test owner should repair expectations while retaining ownership and concurrency checks. Use disposable PostgreSQL checks where real transactions matter. Risk scores should change only when the intended control and its evidence have changed.

6. Treatment plan and completion conditions

Most risks should be reduced through targeted changes. Avoidance is appropriate for an unapproved external provider or an unnecessary public write route. Provider agreements may share some responsibilities, but they do not remove the team's duty to explain its own processing. No risk is recorded as formally accepted.

Proposed stage	Required result	Completion evidence
Before sensitive real-user information	Consistent sessions, approved AI destinations and safe production configuration	R1, R2 and R5 checks pass on the proposed release.
Before broader access	Login and AI abuse limits operate across workers	R3 and R9 repeat-request checks pass.
Before relying on saved records	Ownership, concurrency and recovery are demonstrated	Repaired R6 tests and an R11 recovery exercise.
Before sensitive real-user information	Users can exit locally after a failed save	R4 failed-save and local-cleanup checks.
Next iteration privacy work	Clear retention and export choices	R10 deletion and preview checks.
Before sensitive real-user information	Unsupported evidence and public occupation writes are controlled	R8 negative examples fail safely and R12 unauthorized writes are rejected.

R7 also requires targeted browser and API checks. The backend owner should verify that trusted state-changing requests work while the selected CSRF protection rejects hostile requests. The frontend owner should test dynamic content and proposed browser headers with actual application resources.

These are proposed stages, not promised dates. The team can assign people and dates. Closure needs the changed revision, test result and remaining limitation.

7. Monitoring and contingency decisions

A short review at each iteration boundary should examine authentication failures, denied ownership checks, failed saves, provider destinations and service errors. Deployment changes should trigger a fresh configuration review. AI-provider or payload changes should trigger a fresh privacy review before use.

Choose and record thresholds from normal traffic. Assign someone to check alerts and disable affected paths.

If a suspected incident occurs, contain the affected operation, preserve minimum useful evidence, establish affected records and correct the cause. Re-enable it after focused tests pass. Decide any communication from confirmed facts and applicable obligations. Update the register with the lesson and follow-up action.

8. Overall assessment

Iteration 2 improves saved-work ownership while increasing the amount of personal information and external processing. The strongest next step is to close the existing session, AI disclosure and abuse-control gaps before expanding sensitive use. Recovery and test evidence need equal attention because they show whether protection works when something goes wrong.

This assessment does not certify the hosted system.

Appendix. Primary evidence and review boundary

Paths refer to the repository checkout inspected on 17 September 2026. Supplied examples are not implementation evidence.

ID	Primary source and supported finding
C1	Git snapshots iteration-1-release → 2499736fb8da7f42f579b734dbc34220566b8d81 and current 57d0d782fd9ff4e26f14113b924ac537eb800e3e.
C2	backend/app/main.py:24–66. Mounted routes and CORS configuration.
C3	backend/app/routers/accounts.py:18–106 and frontend/src/services/accountStorage.ts:4–200. Credentials, workspace ownership, revisions and caches.
C4	backend/app/core/security.py:10–96, backend/app/services/auth.py:43–117, backend/app/routers/auth.py:25–44, and legacy auth/user schemas. Session and password findings.
C5	frontend/src/components/account/AccountProvider.tsx:24–165 and frontend/src/components/account/AuthDialog.tsx:25–118. Logout ordering, backup and import choices.
C7	backend/app/core/config.py:48–77, backend/app/services/ai_gateway.py, backend/app/routers/ai.py, backend/app/services/skill_directions.py, backend/app/services/learning_goal.py:17–29 and backend/app/routers/skill_directions.py:16–37. Provider routing and request limits.
C8	Task Assist schemas, model, service, record service and routes under backend/app. Saved exchanges, payloads and user-scoped operations.
C9	backend/app/routers/learning.py and backend/app/services/learning_records.py. User-scoped learning and saved daily briefs.
C10	backend/app/core/config.py:14–46 and backend/app/db/session.py:8–19. Configurable signing, cookie and database settings.
C12	db/schema.sql review-events definition and targeted runtime-source searches. Bounded audit and lifecycle findings.
C13	frontend/src/features/learning-planning/planModel.ts:105–109. Prepared sharing content.
C14	backend/app/routers/occupations.py:23–29, occupation service, repository and model. Unauthenticated legacy-table creation, unchanged from I1.
C15	backend/app/routers/ai.py:46–70. The skill finalizer's verified or response.skills behaviour.

T1: four selected frontend suites covering account storage, learning ownership, learning planning and Task Assist. **T2:** eleven selected backend auth, account, AI and learning suites. **T3:** six observations from the isolated local audit probe. Counts are reported in Section 5. Hosted testing, real-provider evaluation, dependency scans and live PostgreSQL concurrency were outside these results.